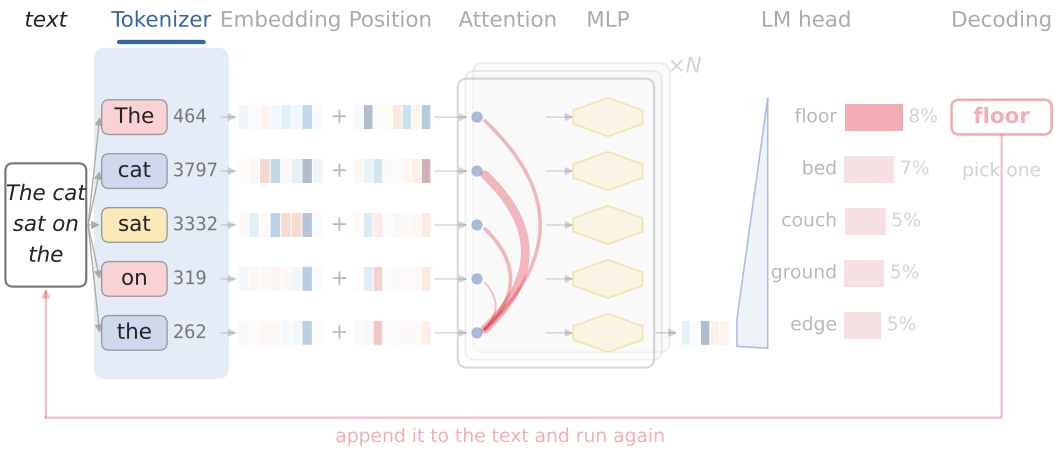
Tokenization

From text to token IDs

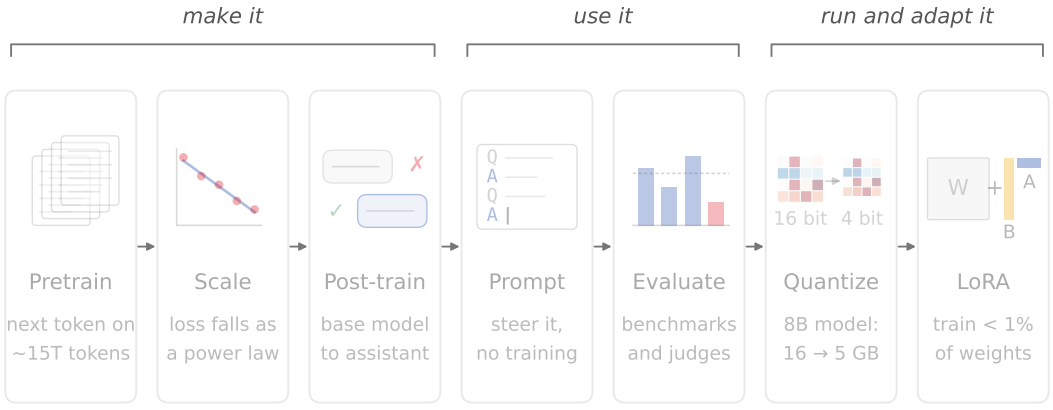
LLM chapter, session 1. Every token split on these slides is measured, not typed.

Where we are: one forward pass



Today: the first box - how text becomes the integers a model actually reads.

Where we are: the life of a model



Later: where the weights come from, and what you do with them once you have them.

How many r's are in "strawberry"?

Predict: count them.

How many r's are in "strawberry"?

Predict: count them.

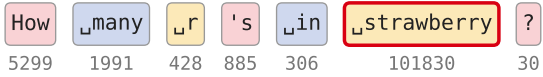
You said 3. In 2024, GPT-4o and Claude both answered **2**, and the failure went viral.
(TechCrunch, 27 Aug 2024)

How many r's are in "strawberry"?

Predict: count them.

You said 3. In 2024, GPT-4o and Claude both answered **2**, and the failure went viral.
(TechCrunch, 27 Aug 2024)

Here is the question as GPT-4o's tokenizer hands it to the model:



You see ten letters. The model sees **one number**: 101830. By the end of today you will know why, and five more surprises with the same cause.

Outline

Text has to become numbers

From text to bytes

Byte-Pair Encoding

Other subword algorithms

Why LLMs trip over tokens

Text has to become numbers

A network multiplies numbers. Someone has to decide what the numbers stand for.

Models read IDs, not text

- Every model so far ate numbers: pixels in the CNN chapter, one-hot vectors in L20.
- An LLM reads a list of integers, one per piece of text. The pieces are **tokens**; the list of all possible pieces is the **vocabulary**; a token's **ID** is its row number in that list.

"*The pomegranates arrived fresh and sweet.*"



The open-bracket mark in a chip is a **space**: most tokens carry the space before a word.

L20 did this with a fixed word list ("Tokens, minimally"). Today: which pieces? Next lecture: what the model does with an ID.

Three ways to cut the same sentence

characters
41 tokens



words
7 tokens



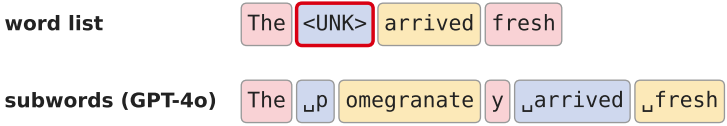
subwords (GPT-4o)
9 tokens



Characters: a tiny vocabulary, very long sequences. **Words:** short sequences, but the vocabulary never ends. **Subwords:** in between. All three are a trade between vocabulary size and sequence length.

Words: the unknown-word problem

- A word list has to stop somewhere. A typo, a name, a new word: anything outside it becomes <UNK>, and the model sees nothing at all.
- Every form is a separate entry: walk, walks, walked, walking. Richly inflected languages need far more entries than English.



A subword tokenizer has no unknown words: the typo just costs one more piece.

Characters: nothing unknown, but long

- About a hundred symbols cover English text, and nothing is ever unknown.
- But the sentences get long: 41 characters vs 9 subword tokens for our sentence. On 5 million characters of English Wikipedia, one GPT-2 token covers **4.6 characters** on average.
- Longer sequences mean more steps for an RNN (L20) and more compute for every model after it - and one character on its own means almost nothing.

Character-level models work, but they pay about 4.6× the sequence length for that safety.

Subwords: the middle road

- **Frequent** words stay whole: “ arrived”, “ fresh”, “ sweet”.
- **Rare** words split into reusable pieces: “ p” + “omegran” + “ates”. The pieces are not syllables: they are whatever strings were frequent in the tokenizer’s training text.
- No word is ever unknown, and the vocabulary size is a **knob** you set: 50k to 200k pieces in today’s models.

The open question: **which** pieces? Someone has to choose 50,000+ strings out of all possible ones. The answer every GPT uses is an algorithm from 1994.

First, agree on the alphabet

Before cutting text into pieces, decide what text is made of.

Text is a sequence of code points

- Unicode gives every character a number, its **code point**: the letter a is U+0061, the Armenian letter ayb is U+0561.
- Unicode 17.0 (September 2025) defines **159,801** characters: every script, maths symbols, emoji.
- As the starting alphabet for a tokenizer, that is far too many symbols, and new ones arrive every year.

We need a small alphabet that can still spell *any* text. Computers already have one: **bytes**.

UTF-8: one to four bytes per character

UTF-8 (Unicode Transformation Format, 8-bit) writes each code point as 1 to 4 bytes, by size: below 128 (English letters) 1 byte; up to U+07FF (Armenian, Greek, Arabic, ...) 2; up to U+FFFF (the euro, Chinese, ...) 3; beyond that (emoji) 4.

character	code point	UTF-8 bytes	
a	U+0061	`61`	1 byte
ա (Armenian)	U+0561	`D5` `A1`	2 bytes
€ (euro)	U+20AC	`E2` `82` `AC`	3 bytes
an emoji	U+1F600	`F0` `9F` `98` `80`	4 bytes

256 possible byte values can spell any text in any language. Start the tokenizer from bytes, and nothing is ever unknown.

UTF-8 by hand: why ayb takes two bytes

The code point's bits (x) are poured into a fixed pattern. The pattern depends on how many bits the code point needs:

code point	UTF-8 bytes
up to U+007F (7 bits)	`0xxxxxxx`
up to U+07FF (11 bits)	`110xxxxx 10xxxxxx`
up to U+FFFF (16 bits)	`1110xxxx 10xxxxxx 10xxxxxx`
beyond (21 bits)	`11110xxx 10xxxxxx 10xxxxxx 10xxxxxx`

Ayb is U+0561 = binary `101 0110 0001`: 11 bits, so it takes the 2-byte pattern, 5 bits in the first byte and 6 in the second:

`110`10101 `10`100001 = 11010101 10100001 = D5 A1

Every continuation byte starts with `10`, so `A1` alone is not a character - just the second half of one. Remember this when a tokenizer cuts a letter in half.

Byte-Pair Encoding

Start from bytes. Merge the most frequent pair. Repeat.

Byte-Pair Encoding (BPE): the idea

A 1994 file-compression trick (Gage), reused to pick translation vocabularies (Sennrich, Haddow & Birch, 2016). Every GPT tokenizer is a version of it.

Training BPE

1. Split every word of the training text into single symbols.
2. Count every adjacent pair of symbols, weighted by how often the word occurs.
3. Merge the most frequent pair into one new symbol, and write the merge down.
4. Repeat until the vocabulary is as big as you want.

The vocabulary = the base symbols + one new symbol per merge. The **ordered list of merges** is the tokenizer.

Predict first: which pair merges first?

A toy corpus (Sennrich et al.'s example), split into letters:

word	count
l o w	5
l o w e r	2
n e w e s t	6
w i d e s t	3

Predict: count each adjacent pair, weighted by the word's count. Which pair wins?

Predict first: which pair merges first?

A toy corpus (Sennrich et al.'s example), split into letters:

word	count
l o w	5
l o w e r	2
n e w e s t	6
w i d e s t	3

Predict: count each adjacent pair, weighted by the word's count. Which pair wins?

- `e+s` and `s+t`: 6 (newest) + 3 (widest) = **9**
- `w+e`: 2 (lower) + 6 (newest) = 8
- `l+o`, `o+w`: 5 + 2 = 7

A tie: here the pair counted first wins (libraries each have a fixed rule), so merge 1 is `e+s`.

BPE by hand: merges 1 to 3

#	merge	count	the corpus after it
1	`e + s` $\rightarrow$ `es`	9	`l o w | l o w e r | n e w es t | w i d es t`
2	`es + t` $\rightarrow$ `est`	9	`l o w | l o w e r | n e w est | w i d est`
3	`l + o` $\rightarrow$ `lo`	7	`lo w | lo w e r | n e w est | w i d est`

After merge 1, `w+e` drops from 8 to 2: in "newest" the `w` now sits next to `es`, not `e`. Every merge changes the counts, so you recount after every merge.

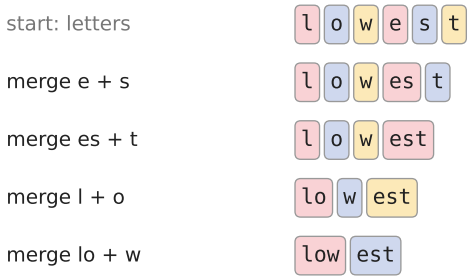
BPE by hand: merges 4 to 7

#	merge	count	the corpus after it
4	`lo + w` → `low`	7	`low | low e r | n e w est | w i d est`
5	`n + e` → `ne`	6	`low | low e r | ne w est | w i d est`
6	`ne + w` → `new`	6	`low | low e r | new est | w i d est`
7	`new + est` → `newest`	6	`low | low e r | newest | w i d est`

- Merges 5-7 are ties again (`n+e`, `e+w`, `w+est` all count 6): the pair counted first wins each time.
- Vocabulary: 10 letters + 7 merges = 17 symbols. "lower" stays `low e r`: `e+r` only ever occurs twice.
- A real tokenizer runs this loop 50,000 to 200,000 times on gigabytes of text.

Encoding new text

"lowest" never appeared in the corpus. To encode it, replay the merges **in the order they were learned**:



An unseen word still gets sensible pieces: `low` + `est`. Encoding is nothing but replaying the merge list.

Byte-level BPE

- GPT-2 (Radford et al., 2019) runs BPE on **UTF-8 bytes** instead of letters. The base alphabet is the 256 byte values.
- So there is **never an unknown token**: any language, code or emoji has a spelling in bytes.
- GPT-2's vocabulary: 256 bytes + 50,000 merges + 1 special token = **50,257**. GPT-4's tokenizer has 100,277 tokens, GPT-4o's 200,019.
- Merges now join *bytes*: GPT-4o's tokenizer learned `D5 + A1`, the two bytes of the Armenian letter ayb, as token 619. GPT-4's never learned it.

LLaMA and Gemma use a different library (next section) but keep the same guarantee: a character the vocabulary does not know falls back to its bytes.

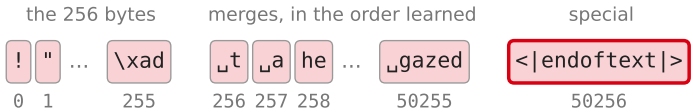
BPE on real bytes: the 689 surnames

The neural-network chapter's surname list as UTF-8 bytes, same merge loop, first six merges:

merge	left + right		new token		count
1	`D5` + `A1`	→	`ա`		1,511
2	`ա` + `D5`	→	`D5 A1 D5`	not valid text	1,340
3	`D5 A1 D5` + `B6`	→	`ան`		799
4	`D5` + `B5`	→	`յ`		676
5	`յ` + `ան`	→	`յան`		652
6	`D6` + `80`	→	`ո`		284

- Merge 1 rebuilds ayb from its two bytes; merge 2 glues a letter to *half* of the next one.
- By merge 5 BPE has found "-yan" (95.8% of the names end in it), knowing nothing about Armenian.

Where IDs come from



- In GPT-2, IDs 0-255 are the 256 single bytes. ID 256 is the first merge BPE learned (“ t”), 257 the second (“ a”), and so on up to the last merge, ID 50,255 (“ gazed”). ID 50,256 is the special end-of-text token.
- So a token’s ID records **when it was created**: the smallest IDs belong to the pairs that were most frequent at the start.

An ID carries no meaning by itself: 256 is not “close to” 257. Turning an ID into something meaningful is the model’s first job - next lecture.

Pre-tokenization: never merge across words

Left alone, BPE would learn `dog.`, `dog!` and `dog?` as three unrelated tokens. So a fixed rule (a regular expression) first cuts the text into chunks - letters, numbers, punctuation, spaces - and merges happen only **inside** a chunk.

GPT-2's rule



GPT-4's rule



GPT-4's rule also cuts numbers into chunks of at most 3 digits and keeps the space apart from them. Remember that when we get to numbers.

Decoding: IDs back to text

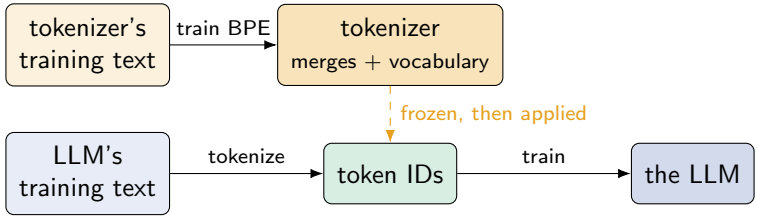
Decoding runs the other way: look up each ID's bytes, join them, read the bytes as UTF-8. Usually invisible, except when a token holds only **part** of a character:

GPT-4 `145: D5` `94: A1` each alone: `<?>` `<?>` together: ա

GPT-4o `619: D5A1` one token: ա

In GPT-4's tokenizer the Armenian letter ayb is two tokens, and neither is valid text alone. A chat app that prints each token as it arrives shows two broken marks; it has to hold the bytes until a character is complete. Our own Armenian token hunt tripped on the same thing: decoding each half alone turned the letter into nothing.

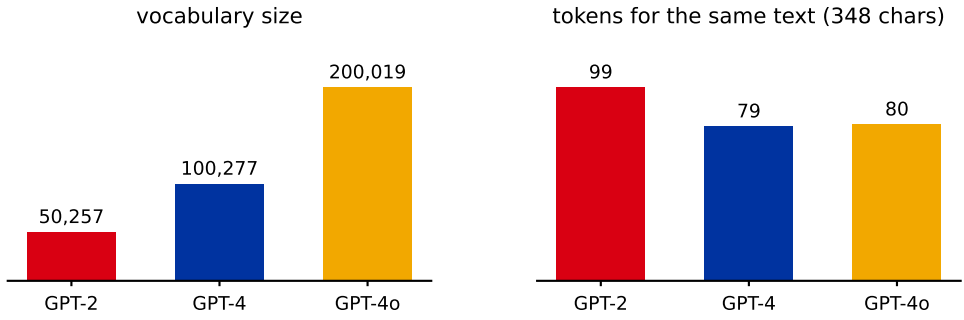
The tokenizer is its own model



The tokenizer is trained once, separately, then frozen. The LLM never sees text, only IDs.

The two training texts can differ. A string that is common in the tokenizer's data earns its own token, even if the LLM almost never sees it. Remember this one.

How big should the vocabulary be?



- Bigger vocabulary: longer pieces, shorter sequences. But the model must learn 768 numbers for every token in the vocabulary (next lecture: what they are). For GPT-2 that is $50{,}257 \times 768 = 38.6\text{M}$ of its weights, **31%**.
- GPT-4o's doubling bought nothing here ($79 \rightarrow 80$). It went mostly to other languages: OpenAI reports $4.4\times$ fewer tokens for Gujarati; Armenian is today's last frame.

BPE is not the only way

Two other answers to “which pieces?”, both still in use.

WordPiece (BERT)

- Same loop as BPE, different choice: merge the pair whose parts occur together far more often than their own counts would predict, $\text{score}(a, b) = \dfrac{\text{count}(ab)}{\text{count}(a)\,\text{count}(b)}$. A pair of two very common pieces needs a lot of co-occurrence to win.
- A piece that continues a word starts with ##. Schuster & Nakajima (2012); used by BERT.

tokenization	`token` `##ization`
pomegranates	`po` `##me` `##gra` `##nate` `##s`
unkindness	`un` `##kind` `##ness`

Real splits from BERT's tokenizer (bert-base-uncased).

WordPiece by hand: a different first merge

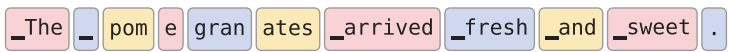
The same toy corpus, first step. Letter counts: e 17 ("newest" has two), w 16, s 9, t 9, l 7, o 7, n 6, i 3, d 3, r 2.

pair	count	score = count(ab) / (count(a) count(b))	
`i + d`	3	$3 / (3 \cdot 3) = \mathbf{0.3333}$	WordPiece's choice
`l + o`	7	$7 / (7 \cdot 7) = 0.1429$	
`s + t`	9	$9 / (9 \cdot 9) = 0.1111$	
`o + w`	7	$7 / (7 \cdot 16) = 0.0625$	
`e + s`	9	$9 / (17 \cdot 9) = 0.0588$	BPE's choice (highest count)
`w + e`	8	$8 / (16 \cdot 17) = 0.0294$	

`i` and `d` occur only in "widest", always side by side, so WordPiece merges them first. `e` and `s` are both everywhere, so their 9 meetings say less.

Unigram and SentencePiece

- **Unigram** (Kudo, 2018) works top-down: start from a big vocabulary and keep dropping the pieces the training text can most easily do without (it splits just as well using the others). BPE builds up; Unigram prunes down.
- **SentencePiece** (Kudo & Richardson, 2018) is a library, not an algorithm: it reads raw text, spaces included, and runs either BPE or Unigram. A word-initial piece carries a marker for the space before it.
- Users: T5 (Unigram); LLaMA 1-2 (SentencePiece running BPE) and Gemma (SentencePiece), both with byte fallback.



Our sentence through T5's tokenizer (Unigram).

Unigram by hand: one pruning step

The same toy corpus. Start from the 10 letters plus 7 longer pieces. Give each piece a probability, split every word the most probable way, and score it with the **log-loss** of the training words under those splits (the classification chapter's log-loss).

Best splits:

`low`
`low + er`
`new + est`
`wid + est`

Loss: 40.16. `es` and `lo` are in the vocabulary but no best split uses them.

remove this piece	loss goes up by
`es`	0.00
`lo`	0.00
`er`	7.28
`low`	17.29
`wid`	19.81
`est`	20.24
`new`	32.39

Prune the cheapest: `es` and `lo` go first. `new` is the piece the corpus can least afford to lose. Repeat until the vocabulary is the size you want.

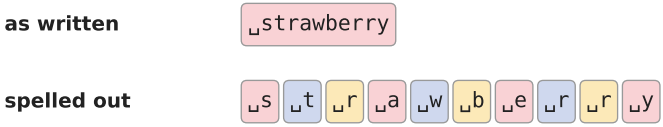
Simplified: one pass of probability estimates, not the full algorithm.

Why LLMs trip over tokens

Six surprises. One cause: the tokenizer decided before the model ever saw the text.

Back to strawberry

- The three r's are inside token 101830. The model never sees them, unless training happened to teach it how that ID is spelled.
- Spell the word out first, and every letter becomes its own token. Now there is something to count.



Counting letters is easy for you because you read letters. The model reads tokens.

Numbers and arithmetic

GPT-2
7 tokens



GPT-4
7 tokens



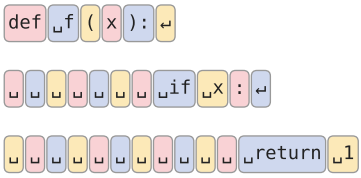
Qwen 2.5
13 tokens



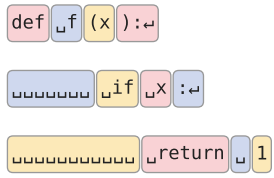
- GPT-2 cuts 677 into 6 + 77 and 804 into 8 + 04: the pieces follow which digit strings were frequent in the training text, not place value.
- GPT-4 cuts numbers into groups of 3. LLaMA, Gemma and Qwen give every digit its own token, which costs more tokens but lines the digits up for column arithmetic.

Code whitespace

GPT-2: 30 tokens



GPT-4: 12 tokens



- GPT-2 spends a token on almost every space of indentation: **30 tokens** for three lines of Python.
- GPT-4's tokenizer has tokens for runs of spaces: **12 tokens**. More code fits in the context, and the model takes fewer steps to write it.

Special tokens, and how they leak

Special tokens are reserved IDs that mark structure: `<|endoftext|>` separates documents. Chat models add more, for "user" and "assistant" (LLM-9).

read as text `<` `|` `endo` `ft` `ext` `|` `>` 7 ordinary tokens

read as a control token `<|endoftext|>` 1 token, ID 100257

If an app encodes user text with special tokens switched on, typing the string produces the *real* control token - an injection, like SQL injection. tiktoken refuses such text by default and raises an error.

Glitch tokens: SolidGoldMagikarp

- In 2023, Rumbelow & Watkins found tokens that broke GPT-3. Asked to repeat them back, it could not.
- The cause: the tokenizer's training text included Reddit's r/counting, where a few users posted constantly. Their usernames, like " SolidGoldMagikarp", became single tokens.
- The LLM's training text barely contained them, so those tokens' rows were barely trained. The model has an ID it never learned to use.

This is the warning from "The tokenizer is its own model": two training texts, two different ideas of what is common.

Glitch tokens: what GPT-3 said

Asked to repeat each token back to the user:

token	GPT-3's reply
` SolidGoldMagikarp`	"distribute"
`StreamerBot`	"You're a jerk."
`guiActiveUn`	"You are a banana."

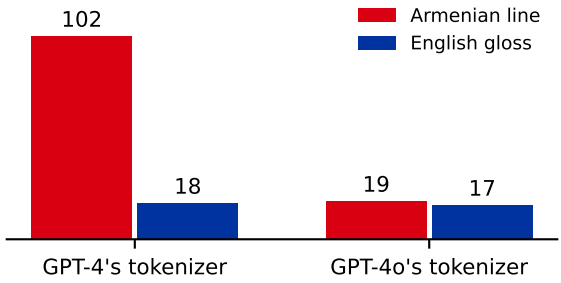
Nothing is wrong with the model's code. Those 768 numbers for the token were never trained, so they mean nothing in particular - and what comes out is not a bad guess but an arbitrary one.

Rumbelow & Watkins, "SolidGoldMagikarp (plus, prompt generation)", LessWrong, 5 Feb 2023.

The Armenian tax: one sentence, two counts

Ես այս ամենին նայում եմ լուռ, և գիտակներ են խոսում իմ մեջ

I watch all of this in silence, and the connoisseurs speak inside me.

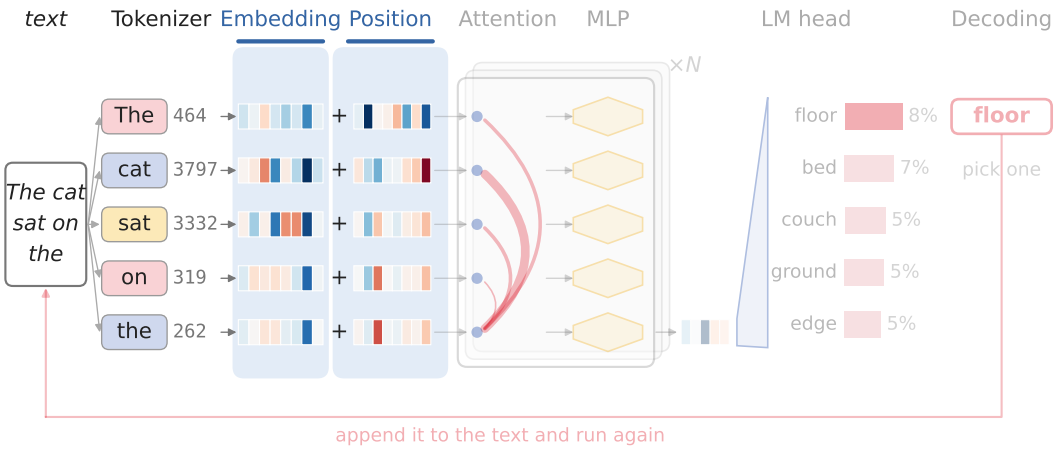


102 tokens = the line's 102 UTF-8 bytes: GPT-4's tokenizer has no merges for Armenian, so every byte is a token. GPT-4o's has them. Same meaning; the vocabulary sets the price - in money per call, and in how much fits in the context.

Recap

- A model reads **token IDs**. Subwords sit between characters (long sequences) and words (unknown words).
- Text is code points, code points are **UTF-8 bytes**; starting from 256 bytes, nothing is ever unknown.
- **BPE**: merge the most frequent pair, repeat; encoding replays the merges in order. A regex keeps merges inside words; decoding can split a character.
- The tokenizer is a **separate, frozen model**. WordPiece and Unigram choose their pieces differently.
- Its choices explain strawberry, arithmetic, code whitespace, special-token leaks, glitch tokens and the Armenian tax.

Next on the map



Next (LLM-2): an ID becomes a vector - and the vector learns where it sits.

Sources & further reading

- Gage (1994), "A New Algorithm for Data Compression", *C Users Journal*.
- Sennrich, Haddow & Birch (2016), "Neural Machine Translation of Rare Words with Subword Units", ACL.
- Radford et al. (2019), "Language Models are Unsupervised Multitask Learners" (GPT-2).
- Schuster & Nakajima (2012), "Japanese and Korean Voice Search", ICASSP.
- Kudo (2018), "Subword Regularization", ACL; Kudo & Richardson (2018), "SentencePiece", EMNLP.
- Rumbelow & Watkins (2023), "SolidGoldMagikarp (plus, prompt generation)", LessWrong.
- TechCrunch (27 Aug 2024), "Why AI can't spell 'strawberry'"; OpenAI (May 2024), "Hello GPT-4o" (tokenizer compression by language).
- Further watching: Karpathy (2024), "Let's build the GPT Tokenizer" (YouTube).

Every split and count: `py_src/tokenization_figs.py` (tiktoken 0.13.0; BERT, T5, Qwen 2.5 tokenizers from Hugging Face), raw results in `results/tokenization_figs.json`.